Running goldsluice and developing an algorithm

Developer quick-sheet · from a Python idea to a fast Rust kernel · version 1, 27 October 2026

`goldsluice` is the fast half of the project: pulse-reduction kernels in **Rust**, each with a **Python mirror** that is the readable definition. You develop in Python on ordinary arrays, then port the same computation to Rust; a test makes sure the two never disagree. It lives in `goldsluice/` (start with its `README.md`; `PORTING.md` is the reference for the port). One algorithm exists, `peak`: the largest ADC value and the largest current above the baseline of each trace.

1. One-time setup

You need **Rust**, **Python 3.9 or newer** and a Python environment (the `darkmatter_cli_env` of the student guides is fine).

- **Rust:** install from https://rustup.rs. On **Windows** also install the Visual Studio *Build Tools* with the workload "Desktop development with C++" (it supplies the linker); then run `rustup-init.exe` with the defaults. On macOS the Xcode command line tools are enough (`xcode-select --install`); on Linux a C compiler (`gcc`). Check in a **new** terminal:

```
rustc --version
cargo --version
```

- **Build and install `goldsluice` in your environment** (from the repository, with the environment active):

```
cd goldsluice
python -m pip install maturin pytest numpy
maturin develop --release
python -m pytest tests
cargo test
```

`maturin develop --release` compiles the Rust and installs the package in place (editable: Python files you change take effect at once; **after a change in `src/` run it again**). Always `--release` when you time anything: a debug build is ten times slower or more.

2. Run it

```
from goldsluice import run, compare

out = run("peak", traces, glitch_samples=10, pretrigger_samples=1000, amps_per_adc=1.5e-9)
out["adc_max"]          # largest raw ADC value, one number per trace
out["current_max"]      # largest current above the pretrigger mean, in amps
```

`traces` is an array of shape `(n_events, n_samples)` for **one channel**. From a `goldpan` `Events` that is a loop over its channels:

```
for name in ev.detector.channel_names():
    out = run("peak", ev.trace(name), glitch_samples=10,
              pretrigger_samples=1000, amps_per_adc=AMPS_PER_ADC)
```

Nothing about the readout is assumed: **no parameter has a default** (`amps_per_adc`, the glitch and pretrigger lengths are the readout's, and a wrong silent value would change every result). The input is converted to C-contiguous `float64` (a copy for `uint16` traces: keep a converted array if you call `run` many times). `run(..., backend="python")` runs the mirror instead of the Rust kernel. Bad parameters raise `ValueError`, with the same message from either backend.

3. Time it

Two numbers, on purpose:

```
print(compare("peak", traces, glitch_samples=10,
              pretrigger_samples=1000, amps_per_adc=1.5e-9))
```

```
cargo bench
```

`compare` is what a user sees: the Python call, the conversion, the output arrays, against the mirror (timed on its first 200 traces and scaled, because it is slow). `cargo bench` times the Rust kernel alone, with no Python: that is the number

to optimise. `RAYON_NUM_THREADS=1` gives the single-thread time. Reference (20000 traces of 4096 samples, one PC, release build): Rust 29 ms with all cores, 149 ms on one thread, Python 15.7 s.

4. Develop an algorithm: the worked example `max_time`

The time of the largest sample (as `goldpan.algorithms.max_time`). The steps are the same for every algorithm. Replace `max_time` by your name.

Step 1. Write the mirror, `python/goldsluice/reference/max_time.py`, in the *kernel subset*: one trace in, plain loops, no numpy inside `kernel`.

```
NAME = "max_time"
OUTPUTS = ("max_time_s",)

def problem(n_samples, sample_period_s):
    if not sample_period_s > 0:
        return "sample_period_s must be positive"
    return None

def kernel(trace, sample_period_s):
    best = 0
    for i in range(1, len(trace)):
        if trace[i] > trace[best]:
            best = i
    return (best * sample_period_s,)
```

Also in the module: `cases()`, the standard inputs `[(traces, params), ...]` (an ordinary case, an edge case such as two equal maxima, a trap) and `bad_cases()`, inputs that must be refused. Register it: `from . import max_time, peak` and `ALL = [peak, max_time]` in `reference/__init__.py`.

Step 2. Develop and check it in Python alone. It works at once, with no Rust:

```
run("max_time", traces, backend="python", sample_period_s=0.8e-6)
```

Compare it with an existing function (here `goldpan.algorithms.max_time`) and fix the idea in Python, where a mistake is cheap. Do not write Rust before this is right.

Step 3. Port it, `src/max_time.rs`: the same lines in Rust (the table in `PORTING.md` has the equivalents).

```
pub struct MaxTimeParams { pub sample_period_s: f64 }

pub fn problem(_n_samples: usize, p: &MaxTimeParams) -> Option<String> {
    if p.sample_period_s.is_nan() || p.sample_period_s <= 0.0 {
        return Some("sample_period_s must be positive".to_string());
    }
    None
}

pub fn max_time_one(trace: &[f64], p: &MaxTimeParams) -> f64 {
    let mut best = 0;
    for i in 1..trace.len() {
        if trace[i] > trace[best] { best = i; }
    }
    best as f64 * p.sample_period_s
}
```

Add `max_time_many` (the loop over traces with `traces.par_chunks(n_samples)`) and a unit test, copying `src/peak.rs`. In `src/lib.rs` add `pub mod max_time;`, a `max_time_py` function that copies `peak_py` (it checks `problem`, then calls the kernel with the GIL released) and one `m.add_function(wrap_pyfunction!(max_time_py, m)?)?;` line. Add the kernel to `benches/kernels.rs`.

Step 4. Build and test.

```
maturin develop --release
python -m pytest tests
```

```
cargo test
cargo clippy --all-targets -- -D warnings
cargo fmt
```

`tests/test_parity.py` needs nothing from you: it finds the new algorithm and checks that it has both versions, that they agree on `cases()` to a relative `1e-12`, and that both refuse `bad_cases()` with the **same message**. A failure names the algorithm and the output.

Step 5. Time it and open the pull request. `compare("max_time", ...)` and `cargo bench`; put both numbers, and the checklist of `PORTING.md`, in the pull request, with a line under `[Unreleased]` in `CHANGELOG.md`. The pull request goes into `develop` like any other (`AGENTS.md`); a workflow builds and tests it on Windows, macOS and Linux.

5. The traps the example met

Trap	What happens
A float in an error message	Python writes `0.0` and `1e-06`, Rust writes `0` and `0.000001`: the two messages differ and the parity test fails. Name the parameter and the rule, with no number.
`not x > 0` **against** `x <= 0.0`	They differ for NaN, which the first refuses and the second lets through. In Rust test `x.is_nan()` as well (clippy refuses `!(x > 0.0)`). Put a NaN in `bad_cases()`.
Order of a sum	Floating-point sums depend on the order. Sum front to back in both; the test allows `1e-12` relative, which is rounding and not a different formula.
Ties and NaN in `max`	Keep the first of equal maxima in both (a strict >). Rust's `f64::max` ignores NaN, Python's `max` does not: decide, and test it.
Forgetting to rebuild	A change in `src/` does nothing until `maturin develop --release` is run again; a change in the mirror takes effect at once.
Timing a debug build	`maturin develop` without `--release` is ten times slower or more.

When an algorithm is naturally array-shaped (an FFT, a fit), `PORTING.md` ends with what to do: ask before you choose that route.